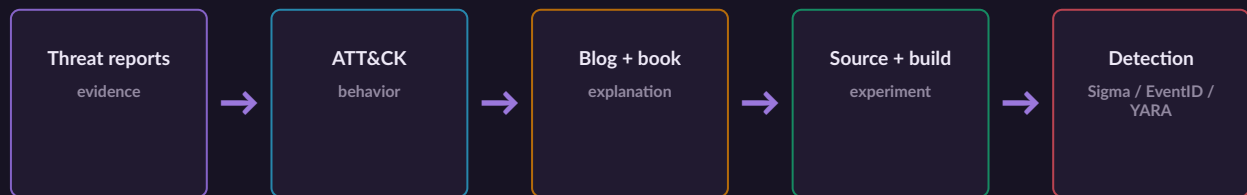


TECHNICAL PAPER + PRACTITIONER MINI-TEXTBOOK

PEEKABOO

From Security Knowledge to Executable, Detectable Adversary Simulations

A GPU-enriched, evidence-aware system that connects an authored malware research corpus and book manuscript to threat intelligence, MITRE ATT&CK, compiled laboratory artifacts, and defender-facing detection coverage.



Zhassulan Zhussupov (@cocomelonc)

Peekaboo project paper

August 2026

Project snapshot: a63a99e

DEF CON 34 workshop edition

FRONT MATTER

Purpose and Scope

Peekaboo is a local-first security research system built around a simple thesis: knowledge becomes operationally valuable when a reader can move from a claim, to the behavior it describes, to an inspectable implementation, to a reproducible artifact, and finally to the telemetry and detections that should observe it.

Ethical and operational scope. The project contains dual-use malware-development examples and can compile native binaries. It is intended for controlled laboratories, detection engineering, purple-team exercises, research, and education. It is not a hardened production control plane, not an authorization to test third-party systems, and not a substitute for legal approval, isolation, monitoring, or professional judgment.

Abstract

Security knowledge is fragmented across prose, source repositories, threat reports, behavioral taxonomies, detection rules, and laboratory tooling. Each representation is useful, but the transitions between them are often manual and lossy. Peekaboo addresses this integration problem. It indexes a first-party corpus of 217 blog documents and 103 book chapters, discovers 191 source modules, enriches the corpus on a local GPU with embeddings and constrained language-model outputs, ingests evidence from HTML and PDF threat reports, format-checks and normalizes ATT&CK mappings, selects related authored implementations, compiles isolated copies of source files, and joins each campaign stage to a local artifact map built from 4,802 Sigma rule associations.

The principal architectural decision is to separate a GPU enrichment plane from a CPU serving plane. Expensive probabilistic work is performed ahead of time and persisted in SQLite. The dashboard and terminal interfaces then render precomputed briefs, mappings, report evidence, and campaign graphs without requiring live generation. A single cached query embedding remains optional for semantic search. This design makes conference and classroom demonstrations deterministic, fast, privacy-preserving, and resilient to network failure.

This paper documents the system as implemented in the August 2026 snapshot, explains the reasoning behind its components, reports measurable corpus and readiness results, positions the project relative to ATT&CK, Atomic Red Team, Caldera, Malpedia, Sigma, and breach-and-attack simulation platforms, and describes limitations that remain. The central contribution is not a new foundation model or a claim to replace those systems. It is a provenance-preserving knowledge-to-practice workflow tailored to an authored technical corpus.

Keywords: threat-informed defense, adversary emulation, malware research, detection engineering, MITRE ATT&CK, Sigma, Malpedia, local LLM, GPU precomputation, reproducible security education.

Terminology: training versus enrichment

This distinction matters. Peekaboo does **not** fine-tune or train the Qwen or Nomic base models. It uses already trained local models for inference. The GPU computes document embeddings, classifications, ATT&CK extractions, evidence-backed report mappings, and compact briefs. Those outputs are persisted in the database. In this paper, *GPU-enriched*, *precomputed*, and *baked* describe that process. Calling it model training would be technically incorrect.

Snapshot and reproducibility

Measurements were taken on 1 August 2026 from commit `a63a99e6eae3e1c230db1625d0230c885879990b`. The local database was 21,360,640 bytes and passed both SQLite `quick_check` and `integrity_check`. The reference enrichment host exposes an NVIDIA L20 with 46,068 MiB of memory. Models recorded in the current enrichment tables include `nomic-embed-text`, `qwen3:1.7b`, and `qwen3:14b`. Historical tag rows also record `qwen3:17b`.

CONTENTS

Table of Contents

1. THE INTEGRATION PROBLEM 6

1.1 Fragmented security knowledge 6

1.2 The knowledge-to-practice thesis 7

2. CORPUS AS PRODUCT INFRASTRUCTURE 8

2.1 Blog, book, and source modules 8

2.2 The operational knowledge model 9

3. SYSTEM ARCHITECTURE 11

3.1 GPU bake, CPU serve 11

3.2 SQLite as a portable evidence store 12

4. GPU ENRICHMENT PIPELINE 13

4.1 Enrichment stages 13

4.2 Guardrails around probabilistic output 13

5. FROM THREAT REPORTS TO ATT&CK 15

5.1 HTML and PDF ingestion 15

5.2 Evidence, confidence, and validation 16

6. FROM ATT&CK TO AUTHORED IMPLEMENTATIONS 17

6.1 Module selection and sophistication 17

6.2 Read-only source and isolated builds 18

7. DETECTION ENGINEERING LOOP 19

7.1 Sigma-derived Artifact Map 19

7.2 Campaign detection overlay 19

8. HUMAN INTERFACES 21

8.1 Minimal dashboard 21

8.2 CLI and terminal application 22

9. WORKED CAMPAIGN STUDY 24

10. EVALUATION AND READINESS EVIDENCE 26

11. RELATED WORK AND DIFFERENTIATION 28

12. TECHNOLOGICAL CONTRIBUTIONS 30

13. RELIABILITY, SAFETY, AND FAILURE DESIGN 32

14. DEPLOYMENT AND DEMO OPERATIONS 34

15. LIMITATIONS AND RESEARCH ROADMAP 36

CONCLUSION 38

APPENDIX A. COMMAND REFERENCE 39

APPENDIX B. DATA MODEL 41

APPENDIX C. GLOSSARY 43

REFERENCES 44

CHAPTER 1

The Integration Problem

Why more information does not automatically produce better security decisions, and why the transitions between representations deserve first-class engineering.

LEARNING OBJECTIVES

- Distinguish a security knowledge base from an operational security workflow.
- Identify where provenance is lost between prose, ATT&CK, code, and detections.
- Understand Peekaboo's central unit of value: an evidence-linked, buildable, detectable behavior.

1.1 Fragmented security knowledge

A mature security team already has information. The difficult problem is preserving meaning while moving that information into practice.

A threat report may describe a process injection sequence in prose. ATT&CK assigns the behavior an identifier and tactic. A research article explains a particular implementation. Source code makes the implementation inspectable. A compiler turns it into a laboratory artifact. Endpoint telemetry records the resulting processes, registry writes, image loads, or network behavior. Sigma rules describe patterns over those observations. Each layer answers a different question:

Intelligence What was observed, by whom, and in which campaign context?	Technique Which reusable behavior does the observation represent?	Implementation How can the behavior be studied as source and reproduced in a lab?
Artifact What concrete file or runtime action can be inspected and tested?	Detection Which telemetry and rules should observe the behavior?	Decision Where is coverage strong, weak, or unsupported by evidence?

Typical workflows keep these layers in separate tools. Links are copied into tickets, ATT&CK IDs are entered manually, source repositories are searched by memory, binaries are compiled ad hoc, and detections are reviewed in another interface. The result is not merely inconvenience. It creates four technical losses:

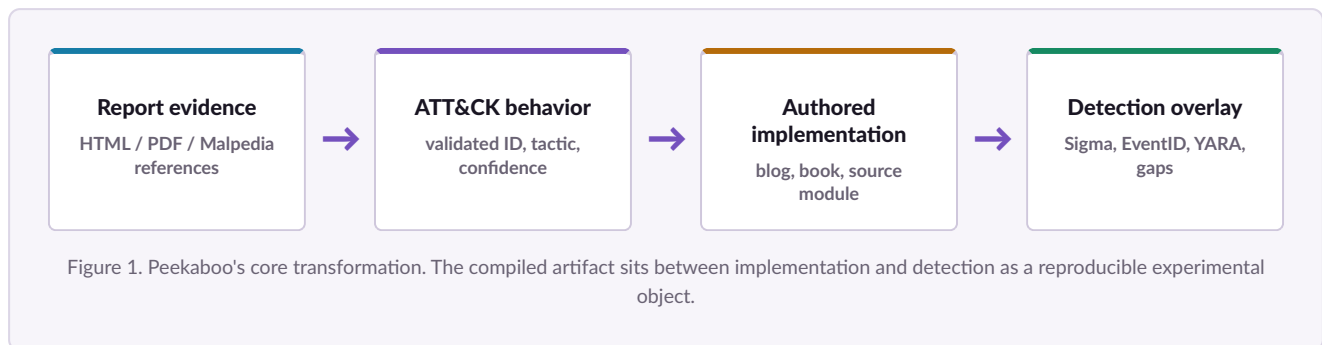
1. **Context loss.** An ATT&CK ID without its evidence sentence can become a weak or incorrect assertion.
2. **Implementation loss.** A taxonomy label does not show which API sequence, platform, compiler, or artifact realizes the behavior.

3. **Detection loss.** A test that executes successfully may still teach nothing about what defenders should observe.
4. **Reproducibility loss.** A live web dependency, a missing compiler, or an unrecorded source choice can make the same exercise behave differently on stage or in class.

1.2 The knowledge-to-practice thesis

Evidence + Behavior + Implementation + Detection = Operational knowledge

Peekaboo treats those transitions as the product. The authored blog and book are not documentation attached to a tool. They are the first-party knowledge substrate from which the tool discovers examples, explanations, source paths, platforms, compilers, and ATT&CK relationships. External intelligence then asks a practical question: *Which part of this authored corpus helps explain and simulate what the report describes?*



This framing also defines what Peekaboo is not. It is not an EDR, SIEM, sandbox, malware corpus distributor, autonomous red-team agent, or universal substitute for ATT&CK. It composes selected capabilities from those domains into a local research and teaching environment.

DESIGN PRINCIPLE

The durable object is not the generated narrative. It is the relationship between evidence, a normalized technique, an inspectable implementation, a build manifest, and defensive telemetry.

REVIEW QUESTIONS

1. Why is an ATT&CK label insufficient as the only output of report analysis?
2. What must be preserved to reproduce a security exercise months later?
3. Which layer in your current workflow owns the link between a detection rule and the code that should trigger it?

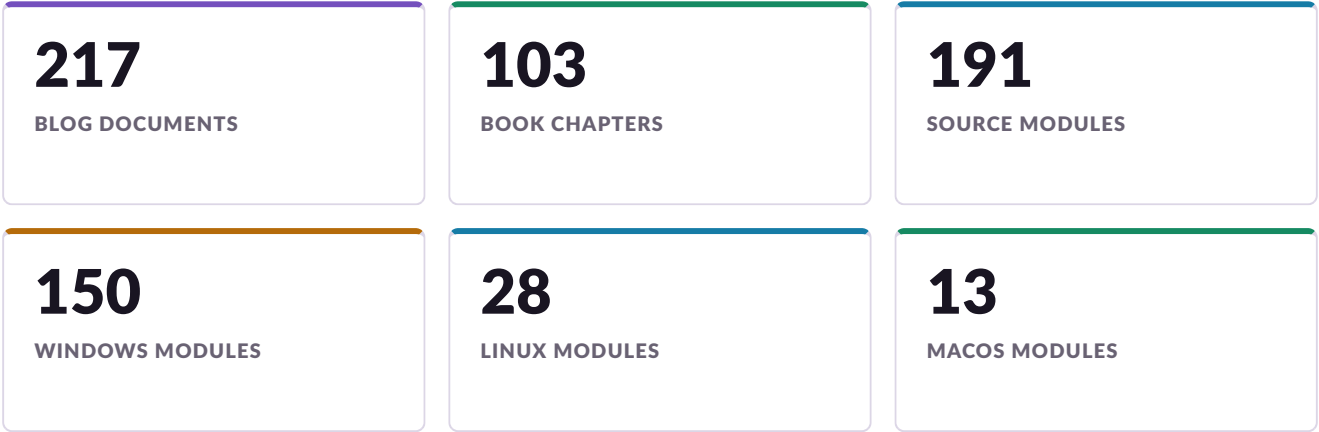
CHAPTER 2

Corpus as Product Infrastructure

The project begins with authored knowledge, then converts editorial structure into machine-usable metadata without taking ownership away from the original repositories.

2.1 Blog, book, and source modules

The August 2026 database contains 320 knowledge documents: 217 blog entries and 103 chapters from the author's malware-development book manuscript. The source discovery layer scans the separate meow repository read-only and currently identifies 191 modules. Of those modules, 150 have a blog link and 177 are marked compilable by the discovery contract.

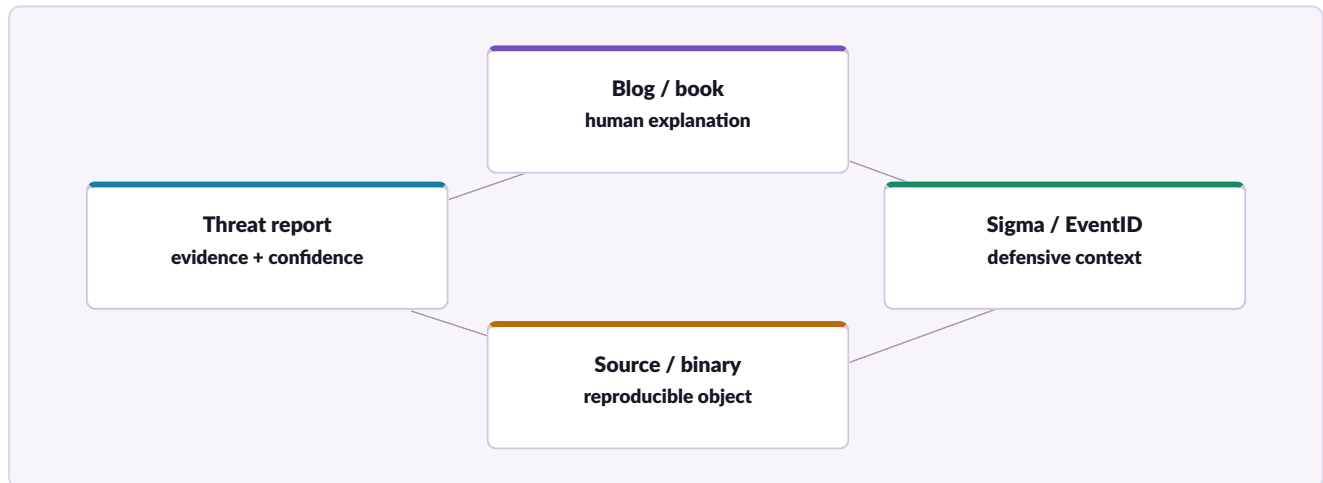


The corpus is unusually valuable because it is vertically integrated. A post title, date, category, source path, compiler, platform, snippet, and blog URL often describe the same experiment. That editorial consistency becomes a useful signal. A later numbered injection article is usually a more specialized example than the first article in the series. A title mentioning an undocumented Native API or callback mechanism carries information about implementation sophistication. Peekaboo can use those signals without maintaining a brittle allowlist of preferred files.

CORPUS CATEGORY	DOCUMENTS	OPERATIONAL VALUE
Evasion	47	Obfuscation, anti-analysis, payload transformation, and bypass-oriented research.
Persistence	38	Registry, scheduled tasks, services, startup mechanisms, and platform-specific persistence.
Injection	28	Classic and advanced Windows process injection implementations.
Tricks	27	Implementation details, native APIs, callbacks, and uncommon execution paths.
Analysis	14	Reverse engineering and observable behavior context.
Linux + macOS	24	Cross-platform persistence, injection, execution, and local research.
Other categories	39	Execution, C2, privilege escalation, exploitation, network, hooking, discovery, and supporting topics.

2.2 The operational knowledge model

Peekaboo stores multiple views rather than flattening everything into one document. A knowledge document is an editorial object. A discovered module is an implementation object. A TTP implementation is a curated relationship between a blog slug and ATT&CK. A report TTP is an evidence-bearing claim about a threat report. An Artifact Map row is a detection-oriented aggregation. A pipeline session is a concrete composition of those elements.



The schema therefore encodes **provenance by separation**. The report quote is not silently merged into the blog summary. The implementation link is not inferred every time a card renders. The compiled binary is not treated as the source of truth for its ATT&CK mapping. Each layer retains its own origin and can be re-evaluated independently.

CURATED AND GENERATED RELATIONSHIPS

Not every relationship has the same trust level. The current database contains 145 curated TTP implementation rows covering 55 distinct ATT&CK IDs and 144 distinct blog posts. Separately, the GPU has produced 287 source-level TTP extraction rows. Curated mappings provide editorial precision; generated mappings provide breadth. Keeping both allows the interface to label where a relationship came from instead of pretending that all tags are equally authoritative.

PRACTICAL LESSON

Do not build a single undifferentiated “AI knowledge table.” Store source identity, model identity, timestamps, raw output where needed, normalized fields, and human-curated mappings separately. This makes replacement and audit possible.

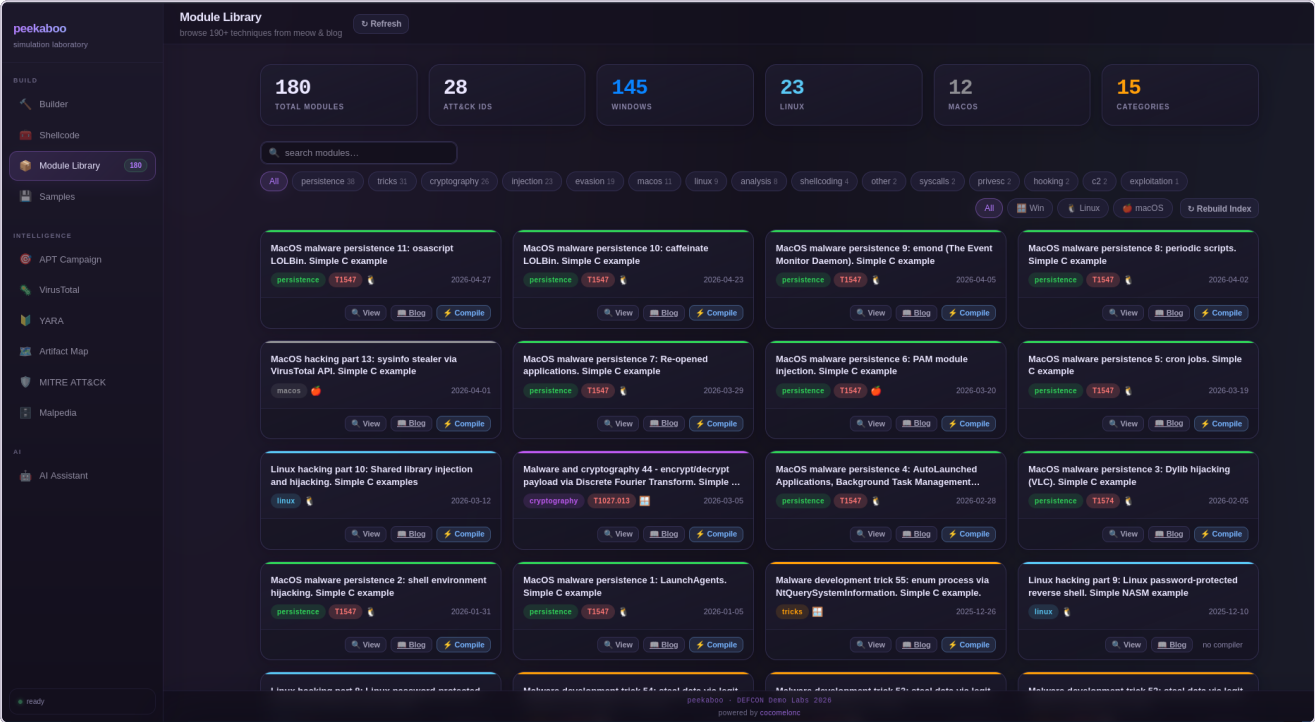


Figure 2. The Module Library presents discovered source as a working catalog with platform, category, ATT&CK, blog, source, and compilation actions. Screenshot counts reflect the UI state when captured; evaluation tables in this paper use the later database snapshot.

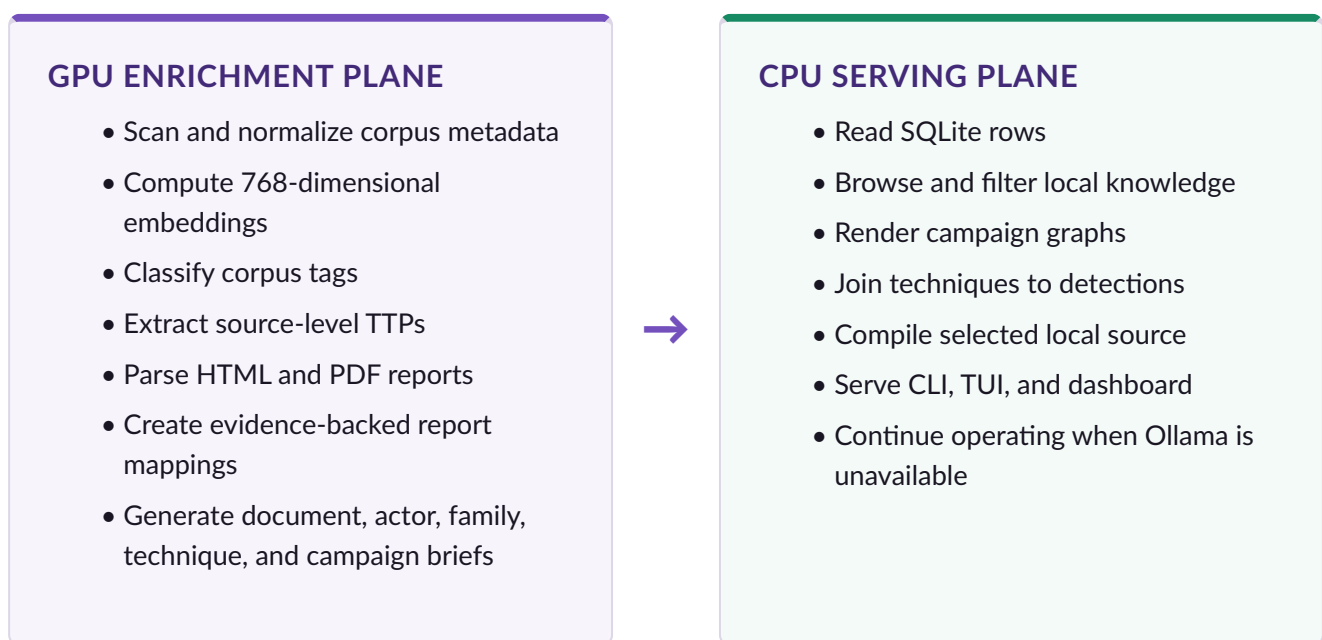
CHAPTER 3

System Architecture

A portable database is the contract between an expensive enrichment environment and a deliberately lightweight serving environment.

3.1 GPU bake, CPU serve

Peekaboo divides work by lifecycle, not by fashion. Embedding hundreds of documents, reading reports, producing constrained JSON, and creating thousands of short briefs are batch operations. They change when the corpus or intelligence changes. Rendering a card, browsing ATT&CK, opening a campaign graph, or showing a brief is interactive serving work. It should not wait for generation.



The principal runtime exception is semantic search. A new query can require one lightweight embedding request. The resulting vector is cached in `query_embeddings`, so repeated searches cost no model round-trip. If the embedding provider is unavailable, semantic search can report a degraded state while all precomputed material remains available.

WHY THIS SPLIT MATTERS

Deterministic demonstrations

A conference session does not depend on Wi-Fi, a live PDF host, or a model completing on time.

Privacy

Private book and source content can stay within the local environment and local model host.

Inspectability

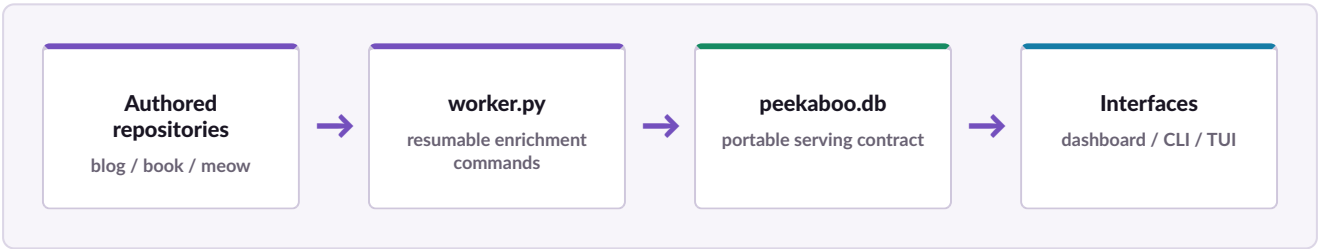
Generated results are rows that can be queried, counted, diffed, backed up, and reviewed.

Cost control

Generation happens when data changes, not every time a user opens the same screen.

3.2 SQLite as a portable evidence store

The database is the deployment boundary. The current schema contains 21 tables and occupies approximately 21 MB. This is small enough to snapshot and copy, yet structured enough to preserve model names, timestamps, raw output, evidence hashes, session manifests, and many-to-one relationships.



SQLite Write-Ahead Logging improves concurrent read/write behavior, but it changes operational copying. A raw copy of the main file while its WAL contains uncheckpointed changes can be incomplete. Peekaboo's documented workflow therefore calls for `PRAGMA wal_checkpoint(TRUNCATE)` before copying, or SQLite's `.backup` command for an atomic snapshot. The operational lesson is broader: **portability is only useful when snapshot semantics are explicit.**

ARCHITECTURE LAYERS

LAYER	MAIN COMPONENTS	RESPONSIBILITY
Discovery	dashboard/discovery.py, dashboard/mitre.py	Read authored repositories, resolve source and blog relationships, infer platform/compiler metadata.
Persistence	dashboard/db.py, SQLite	Schema initialization, idempotent upserts, report evidence, summaries, sessions, and sample records.
Enrichment	worker.py, Ollama APIs	Batch embedding, extraction, summarization, report processing, and readiness counts.
Simulation	pipeline/apt_pipeline.py	Actor/family resolution, report TTPs, module selection, stage ordering, compilation, and detection overlay.
Build	dashboard/compiler.py	Copy read-only inputs into temporary directories, infer libraries, invoke native toolchains, record outputs.
Presentation	Flask dashboard, Cytoscape.js, Rich/Textual CLI	Operational browsing, graph exploration, source viewing, and readiness workflows.

CHAPTER 4

GPU Enrichment Pipeline

The GPU is used as a batch knowledge processor. Its outputs are constrained, persisted, and surrounded by deterministic code.

4.1 Enrichment stages

The worker exposes small subcommands rather than one opaque job. This makes the process resumable, observable, and replaceable. The same source corpus can be re-embedded without re-running report ingestion; campaign briefs can be rebuilt without touching source tags.

STAGE	DEFAULT MODEL	PERSISTED RESULT	CURRENT ROWS
embed	nomic-embed-text	768-dimensional vectors for semantic retrieval	320
tag	qwen3:1.7b	Constrained corpus classification; multiple model generations may coexist	730 total tag rows
ttp	qwen3:14b	Source-level ATT&CK extraction with rationale and confidence	287
summarize	qwen3:14b	Compact what/how/detection briefs for knowledge documents	320
reports	qwen3:14b	Evidence-bearing ATT&CK rows from linked HTML/PDF reports	1,151
sigma	qwen3:14b	Detection brief for each Artifact Map technique	410
actor	qwen3:14b	Threat actor profile brief	979
family	qwen3:14b	Malware family behavioral brief	3,704
apt	qwen3:14b	Completed campaign brief	12

EMBEDDINGS ARE COORDINATES, NOT ANSWERS

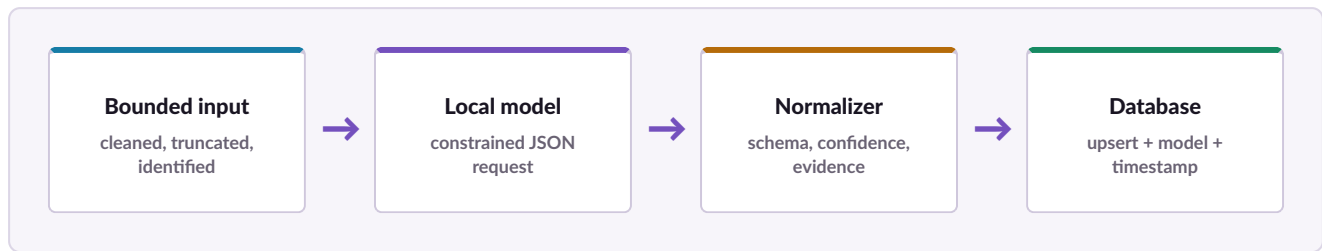
nomic-embed-text maps each document and query into a vector space. Peekaboo uses cosine similarity to rank authored posts against a threat actor or malware family description. The vector does not prove that a post implements the actor's behavior. It proposes a neighborhood for a human or downstream deterministic filter to inspect. This is a crucial semantic distinction: retrieval is a candidate-generation mechanism.

CONSTRAINED GENERATION

For extraction tasks, prompts request strict JSON with a small schema. Code then parses, normalizes, limits, and stores the result. For summaries, the worker applies a contract that strips unwanted formatting and limits structure. This moves complexity from free-form prose toward typed fields that the product can validate and render consistently.

4.2 Guardrails around probabilistic output

The system uses a useful pattern: **probabilistic middle, deterministic edges**. The model interprets prose, but ordinary code controls what enters and leaves that interpretation step.



Rows include model identity and generation time. Report mappings preserve a short evidence excerpt and an evidence hash. Existing rows are skipped by resumable queries, which makes interruption a normal state rather than a failed batch. Raw model output is retained for selected enrichment tables so a malformed or surprising normalized result can be audited.

A MODEL IS NOT AN ORACLE

Format validation does not imply semantic truth. Exact ATT&CK sub-technique validation, report-quote verification, taxonomy versioning, and human review remain necessary. The current report corpus demonstrates both the value and the limits of automated extraction; Chapter 15 describes the remaining hardening work.

WHY LOCAL MODELS FIT THIS PROJECT

- The authored corpus and source code do not need to leave the research environment.
- Batch latency is acceptable because serving latency is decoupled from generation.
- Model replacement does not require changing dashboard contracts when normalized schemas remain stable.
- Multiple model generations can coexist for comparison and migration.
- The database can be shipped to a CPU-only machine for an offline demonstration.

REVIEW QUESTIONS

1. Why should embedding similarity be treated as candidate generation rather than proof?
2. Which fields make a generated mapping auditable?
3. What changes when enrichment and serving have separate failure domains?

CHAPTER 5

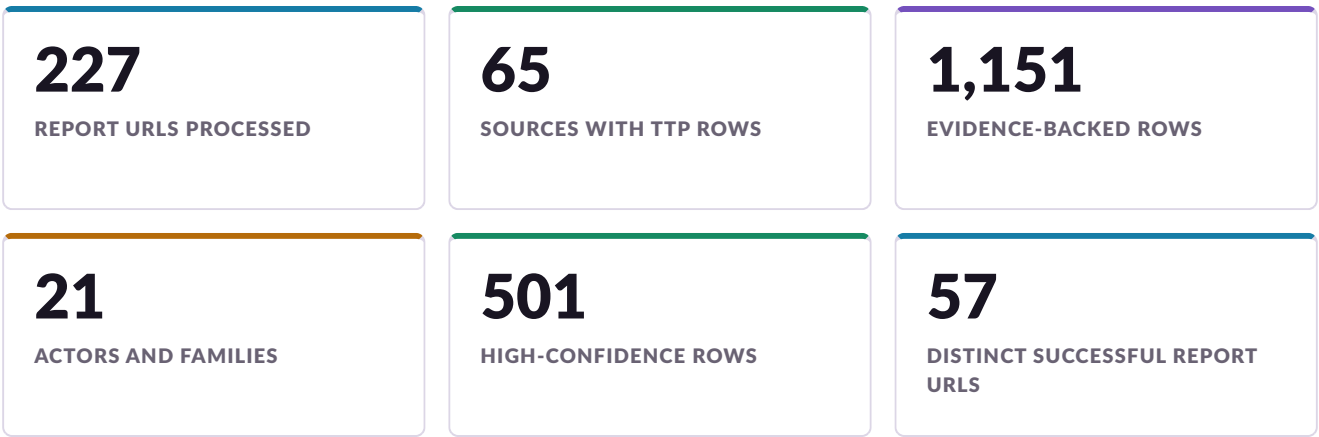
From Threat Reports to ATT&CK

A useful campaign simulation needs report-level evidence, not only an actor label or a regular expression that finds already printed T-codes.

5.1 HTML and PDF ingestion

Malpedia provides actor and family context together with references to analysis reports [3, 4]. Peekaboo uses those references as a curated starting point. The report worker resolves an actor or malware family, selects a bounded number of URLs, downloads each document, identifies HTML or PDF, extracts text, cleans and truncates it, and submits it to a local model for ATT&CK mapping.

This improved an earlier live-only path in three ways. First, PDF bytes are parsed rather than treated as text. Second, behavior described in prose can be mapped even when no literal Txxxx string appears. Third, results are stored before the demonstration, so report hosts, redirects, rate limits, and conference connectivity are no longer in the critical path.



Failure is expected in public report collections. In the measured batch, 71 URLs failed to fetch, 72 produced no TTPs, 11 produced invalid model JSON, and 8 were too short after extraction. These records are not silently retried on every run. Their status is persisted, which allows a curated future pass to focus on broken or low-value sources.

STATUS	COUNT	MEANING	OPERATIONAL RESPONSE
ok	65	At least one normalized evidence-bearing TTP was saved.	Available to offline campaign simulation.
no_ttps	72	Document parsed, but no supported mapping survived normalization.	Keep as processed; review only if strategically important.
fetch_error	71	HTTP, TLS, timeout, 403, 404, or other retrieval failure.	Replace stale links or skip for the curated demo corpus.
llm_error	11	The response was not valid constrained JSON.	Retry with corrected prompt/model only when needed.
too_short	8	Extracted text was below the minimum useful size.	Look for a direct PDF or alternate report.

5.2 Evidence, confidence, and validation

Each report mapping stores the subject type and ID, report URL and title, model, ATT&CK ID, normalized name and tactic, confidence, evidence text, evidence hash, raw output, and extraction timestamp. The evidence excerpt is central: it lets the reader judge whether the mapping reflects the report or merely the model's prior knowledge.

EVIDENCE CONTRACT

A report TTP row without evidence is rejected. Confidence must be one of high, medium, or low. Duplicate evidence for the same technique is removed within a report. Tactics are normalized by code rather than accepted blindly from the model.

The pipeline prefers precomputed report TTPs. If none exist, it can fall back to live report content and local extraction. That fallback preserves basic functionality on a new installation, while demo actors use richer offline data. A fallback is not equivalent to the enriched path: it may have fewer techniques, less evidence, and more network dependence.

REPORT CORPUS USED FOR THE MEASURED DEMO SET

The precomputed database covers 11 actor subjects and 10 family subjects. Actors include APT10, APT28, APT29, APT41, FIN7, Lazarus Group, MuddyWater, OilRig, Sandworm, TA505, and Turla. Families include Agent Tesla, AsyncRAT, Cobalt Strike, Emotet, FormBook, NanoCore, QakBot, Quasar RAT, RedLine Stealer, and Remcos. This is deliberately curated rather than an indiscriminate copy of every Malpedia reference.

TAXONOMY DRIFT IS A DATA ENGINEERING PROBLEM

ATT&CK changes over time. As of this paper, ATT&CK v19 includes important taxonomy changes [2]. A local Artifact Map, a curated implementation table, and a report extraction batch can therefore reflect different snapshots. Validation must record the ATT&CK catalog version, require exact sub-technique membership, and support explicit migration. Accepting a known parent ID for an unknown suffix is convenient for version skew but can admit nonexistent sub-techniques. Peekaboo's current worker still has this parent-fallback behavior, so strict catalog versioning is listed as priority hardening rather than claimed as completed.

CHAPTER 6

From ATT&CK to Authored Implementations

A campaign becomes educational when a technique opens into the author's explanation, source code, build metadata, and a concrete artifact.

6.1 Module selection and sophistication

A naive selector treats every module tagged with the same technique as equivalent. That fails in a corpus containing a long research series. For Process Injection, a classic introductory `VirtualAllocEx` example and a later callback or undocumented Native API technique may share the same high-level ATT&CK label but carry very different demonstration value.

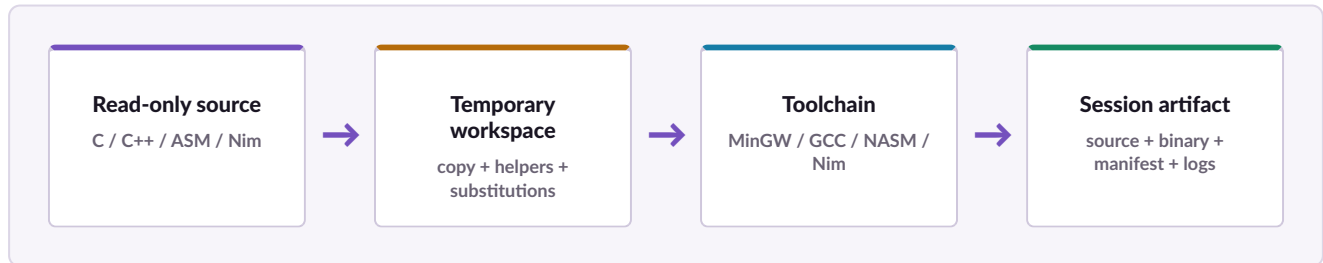
Peekaboo calculates a deterministic base score from tactic alignment, platform suitability, blog linkage, implementation quality, and metadata-derived sophistication. It then performs a weighted choice among high-scoring candidates. A supplied random-number generator makes runs reproducible when desired. Recent session history can reduce repetition, increasing breadth across a demonstration set.

SIGNAL	REASON	EFFECT
Tactic/category alignment	Prevents a loosely tagged source from outranking a module whose category matches campaign intent.	Higher base relevance.
Platform and compiler	The default campaign target is Windows, but the registry also contains Linux and macOS material.	Buildable target preference.
Blog relationship	An implementation with a first-party explanation is more useful for teaching and evidence navigation.	Provenance bonus.
Series position and title	Later posts and advanced API terminology often indicate a deeper implementation.	Sophistication bias.
Recent usage	Repeatedly choosing the same source makes multi-demo coverage shallow.	Diversity penalty.
Top-candidate weighting	Always choosing the maximum becomes repetitive; choosing from all matches becomes noisy.	Controlled variety.

The important point is that sophistication is derived from real corpus metadata rather than a secret hardcoded list. The scoring output is stored with the session stage, so a reader can see why a module was selected and compare it to alternatives.

6.2 Read-only source and isolated builds

The meow repository remains read-only. The compiler layer copies primary and helper sources into a temporary directory, applies build-specific parameter replacements there, invokes the selected toolchain, and snapshots the successful output. This protects the authored repository from accidental mutation and prevents one build's generated state from contaminating another.



Compiler selection is discovered from source extension and target platform. The current module registry contains 70 mingw-gcc, 74 mingw-g++, 33 gcc, 10 NASM, and 4 Nim candidates. The build layer can infer common link dependencies such as Winsock, WinHTTP, IP Helper, Crypt32, Shell32, WinINet, and the portable math library. It also distinguishes Windows executables from DLL-shaped source.

SESSION MANIFEST AS EXPERIMENTAL RECORD

A completed campaign writes copied source, binaries, optional YARA drafts, and `manifest.json`. Each stage records technique, tactic, report evidence, selected module, original blog URL, platform, compiler, source name, selection score, sophistication, detection overlay, output filenames, sizes, and build errors. The manifest is the experiment's bill of materials.

WHY SOURCE COPIES MATTER

A binary hash alone cannot explain which authored implementation, parameters, compiler family, and evidence relationship produced it. The session keeps the source copy and manifest beside the binary so the artifact remains interpretable.

CHAPTER 7

Detection Engineering Loop

Simulation is incomplete until the system shows what defenders can observe and where current coverage is absent.

7.1 Sigma-derived Artifact Map

Sigma is a generic, shareable signature format for log events [5]. Peekaboo parses a local Sigma repository and groups rule metadata by ATT&CK technique. The resulting Artifact Map stores the technique ID, name, tactic labels, rule count, Event IDs, registry keys, process names, command-line indicators, and compact rule metadata.



This is not the same as claiming that every rule is enabled, tuned, or effective in a particular environment. Rule association is *available detection knowledge*. Actual coverage depends on telemetry collection, field mapping, product semantics, exclusions, baseline behavior, and test execution. Peekaboo calls the join a detection overlay, not a production validation result.

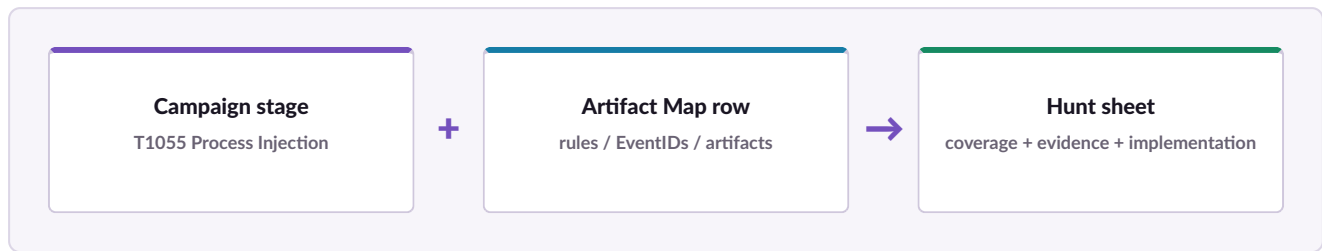
WHY EXTRACT ARTIFACTS FROM RULES?

An analyst examining a simulated stage should not have to open dozens of YAML files to begin hunting. A compact overlay can answer the first questions immediately: Which Event IDs recur? Which process names and command-line fragments are relevant? Are registry paths present? How many rules reference this behavior? The full rule list remains available for inspection.

7.2 Campaign detection overlay

For every selected stage, `agent_detection_overlay()` joins its ATT&CK ID to the Artifact Map. The session then receives stage-level detection fields and a campaign rollup:

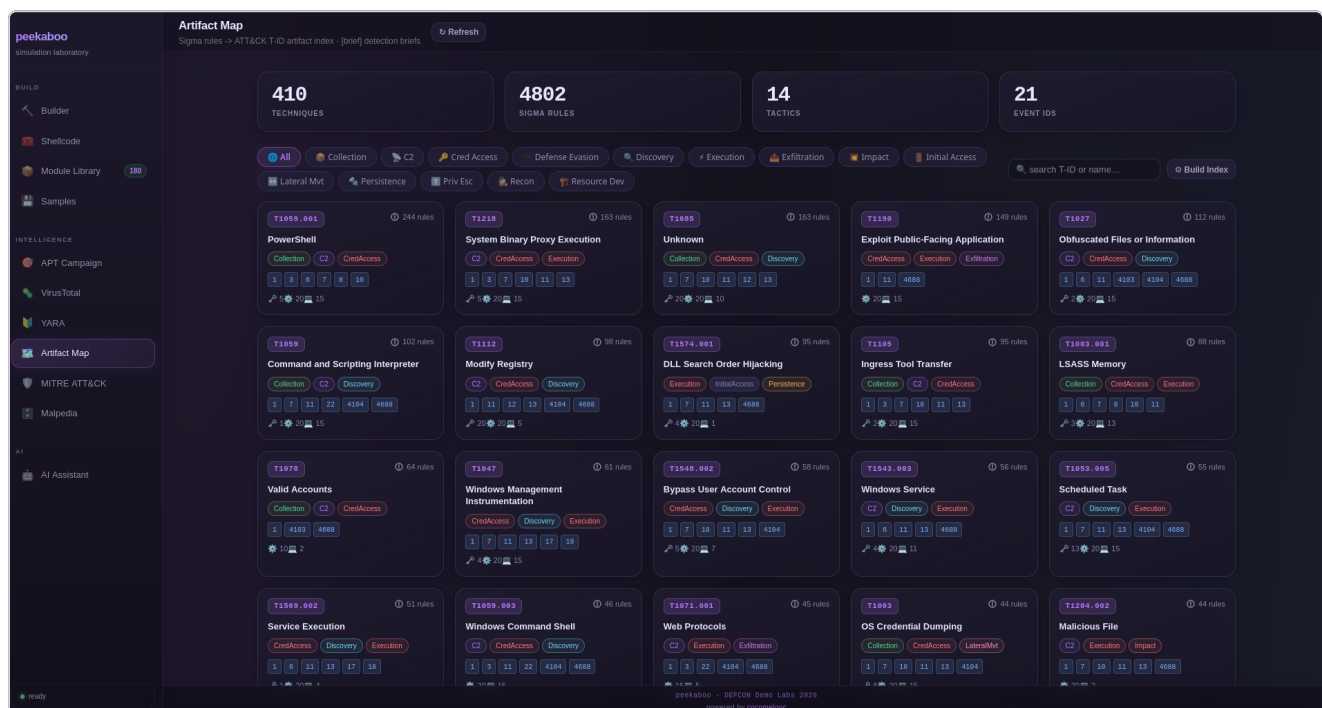
- **Coverage percentage:** stages with at least one mapped detection association divided by total stages.
- **Unique Sigma rules:** deduplicated rule references across the chain.
- **Unique Event IDs:** telemetry identifiers across all mapped stages.
- **Gaps:** stages with no mapped rule association.
- **Generated YARA:** a starter binary rule for compiled blind-spot stages.



Where a compiled blind-spot stage exists, Peekaboo can generate a conservative starter YARA rule from the artifact. YARA describes file or memory patterns through strings and Boolean conditions [16]. The generated rule is intentionally a draft: it closes the workflow loop by producing a reviewable starting point, not by claiming production quality.

PURPLE-TEAM VALUE

The same screen shows why the stage exists, where it came from, how it was implemented, what was built, and what defenders should inspect. This reduces the handoff gap between offensive research and defensive engineering.



CHAPTER 8

Human Interfaces

The dashboard and terminal application expose the same knowledge spine at different levels of density, without turning every feature into a separate product.

8.1 Minimal dashboard

The dashboard is an operational workspace rather than a marketing landing page. Navigation is grouped into build, intelligence, and AI domains. Large tables use compact typography and consistent tabs. Detail panels reveal source, evidence, detection, and briefs only when requested. Long-running tasks stream progress; precomputed data renders immediately.

WORKSPACE	PRIMARY TASK	KNOWLEDGE CONNECTION
Builder	Configure and compile a selected implementation.	Source module, platform, compiler, build history.
Module Library	Browse authored implementation inventory.	Blog explanation, ATT&CK, source, compilation.
APT Campaign	Compose report-derived behavior into a lab campaign.	Evidence, TTPs, graph, binaries, detection.
Artifact Map	Inspect technique-level telemetry and rule coverage.	ATT&CK, Sigma, Event IDs, briefs.
MITRE ATT&CK	Browse groups, authored technique library, and extractions.	Curated and generated corpus mappings.
Malpedia	Search actors and malware families.	Threat context and semantically related posts.
YARA / VirusTotal	Generate local rules or explicitly query external analysis.	Compiled artifacts and file-level inspection.
AI Assistant	Ask contextual technical questions.	Retrieved blog/book context and source snippets.

THE CAMPAIGN GRAPH

The graph is generated deterministically from session data already stored in SQLite. Cytoscape.js, an open-source network visualization library [11], is vendored locally. ATT&CK tactics become lanes; stages follow campaign order; selecting a stage opens report evidence, implementation metadata, source and binary links, Sigma rules, and Event IDs.

Three graph modes answer different questions:

- **Campaign:** What is the ordered threat path?
- **Hunt:** What detection context and gaps exist at each stage?
- **Evidence:** Which report supports the stage, and which authored implementation and artifact realize it?

8.2 CLI and terminal application

The CLI is not merely a script wrapper. It provides machine-readable JSON, explicit offline behavior, safe consent for network uploads, compact campaign views, ATT&CK and artifact queries, Malpedia search, local shellcode analysis, builder workflows, and a terminal UI for navigation. Rich formatting is disabled automatically when output is captured, and dynamic data is escaped before rendering as markup.

The terminal application mirrors the visual language of the dashboard and adds a Monokai-style source viewer. This matters in workshops and remote shells where a browser may be inconvenient. The same knowledge object can be inspected as a dashboard card, a graph detail panel, a CLI record, or highlighted source without duplicating its semantics.

The screenshot shows the Peekaboo terminal application interface. At the top, the header displays 'Threat Research and Detection Engineering Lab' and statistics: 'DB 20.4 MB | 12 campaigns | 206 modules | 410 detections'. The left sidebar contains a 'NAVIGATION' menu with items: 1 Overview, 2 APT Campaigns, 3 Research Library (selected), 4 MITRE ATT&CK, 5 Detection, 6 Threat Intel, 7 Build History, 8 Samples, and 9 Toolkit. The main area is titled 'Research Library [206]' and includes a search bar. Below the title, it says 'Blog code, source paths and ATT&CK mappings'. A table lists research entries with columns: DATE, SLUG, CATEGORY, and ATT. The table contains 20 rows of data. To the right of the table, a source viewer displays the content of the selected entry, 'linux-hacking-11', showing a blog post snippet about Linux hacking and a C example. The bottom of the interface features a command prompt with various shortcuts: '1-9 views / search enter detail s source a assistant b build y yara r reload ? help q quit'.

DATE	SLUG	CATEGORY	ATT
2026-06-17	linux-hacking-11	linux	-
2026-05-28	malware-tricks-58	tricks	-
2026-05-26	malware-tricks-57	tricks	-
2026-05-25	malware-tricks-56	tricks	-
2026-04-27	mac-malware-persistence-	persistence	T15
2026-04-23	mac-malware-persistence-	persistence	T15
2026-04-12	malware-android-3	android	-
2026-04-05	mac-malware-persistence-	persistence	T15
2026-04-02	mac-malware-persistence-	persistence	T15
2026-04-01	malware-mac-13	macos	-
2026-03-29	mac-malware-persistence-	persistence	T15
2026-03-20	mac-malware-persistence-	persistence	T15
2026-03-19	mac-malware-persistence-	persistence	T15
2026-03-12	linux-hacking-10	linux	-
2026-03-05	malware-cryptography-44	cryptography	T10
2026-02-28	mac-malware-persistence-	persistence	T15
2026-02-05	malware-mac-persistence-	injection	T15
2026-01-31	malware-mac-persistence-	persistence	-
2026-01-05	malware-mac-persistence-	persistence	-
2025-12-26	malware-tricks-55	tricks	-

Linux hacking part 11: GOT/PLT hijacking. Simple C example.

Slug: linux-hacking-11
Category: linux
ATT&CK: -
Source: /home/cocomelonc/hacking/meow/2026-06-17-linux-hacking-11/hack.c

[Open blog post](#)

[Brief](#)

Overwrites a function pointer in the GOT to redirect calls to

Figure 4. Peekaboo's terminal application brings dashboard-style navigation, compact intelligence tables, status, and an assistant into an SSH-friendly interface.

Campaign graph: evidence, implementation, and detection in one view

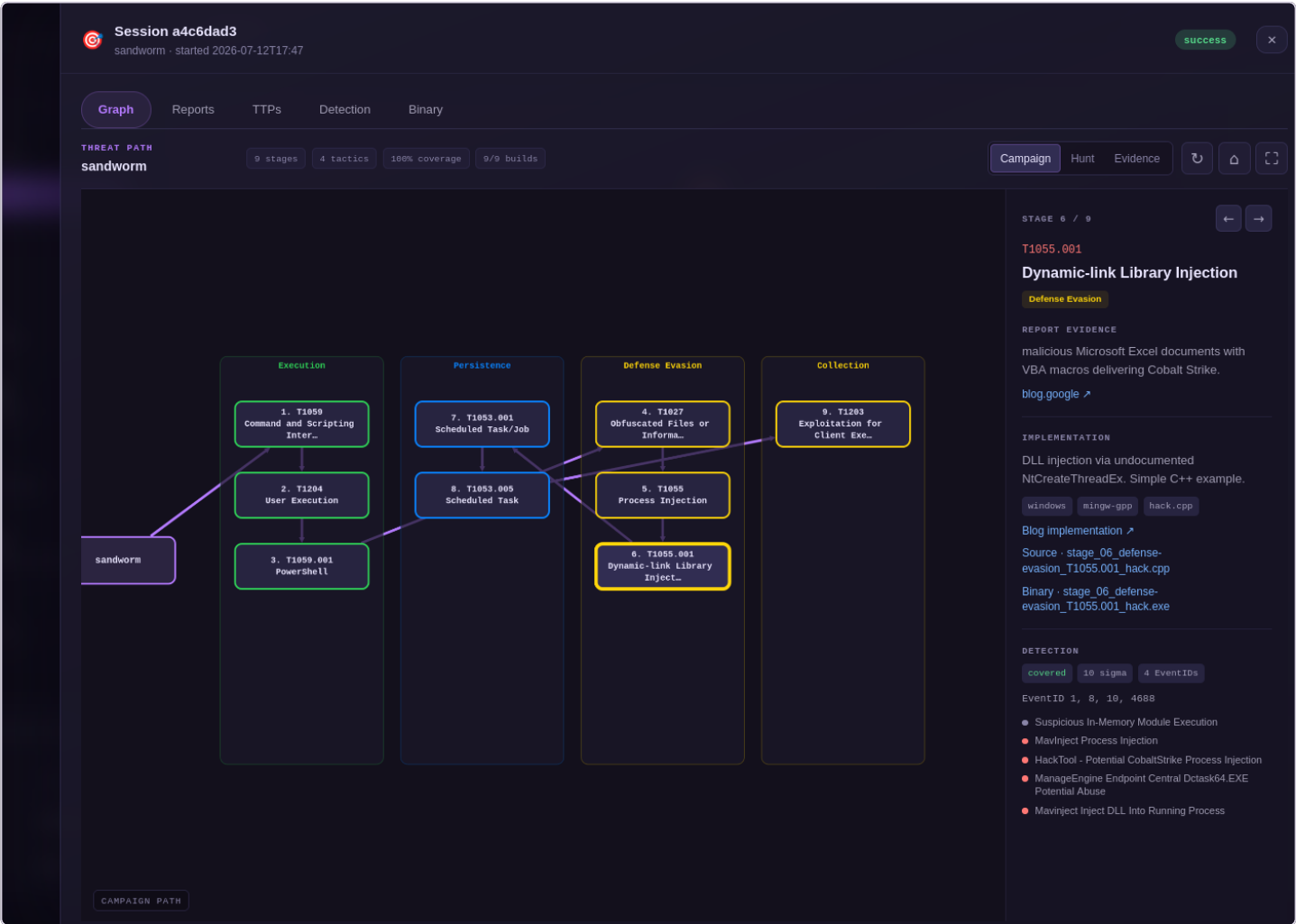


Figure 5. A completed Sandworm session. The selected Dynamic-link Library Injection stage exposes its report evidence, authored blog implementation, compiler and platform, copied source, compiled binary, Sigma associations, and Event IDs. The graph itself is local and requires no model call.

CHAPTER 9

Worked Campaign Study

A step-by-step reading of the Sandworm demo session shows how the layers combine without implying that a small lab chain is a complete reconstruction of a real intrusion.

9.1 Start from a threat subject

The operator selects `sandworm`. Peekaboo resolves the Malpedia actor context and checks for precomputed report TTPs. In the measured report corpus, Sandworm has eight attempted source URLs, three successful report sources, and 42 evidence-bearing report TTP rows. The pipeline does not need to download those reports again for the prepared demo.

9.2 Normalize and bound behavior

The extracted behavior set is ordered by a kill-chain tactic sequence and bounded by the configured stage limit. The measured session `a4c6dad3` contains nine stages across execution, persistence, defense evasion, and collection. A stage is not accepted because an actor is popularly associated with a technique; it carries a report-derived evidence field into the manifest.

9.3 Select authored implementations

For each technique, the selector scores matching local modules and chooses from strong candidates. In the displayed session, examples include PowerShell-related execution, payload obfuscation, Process Injection, Dynamic-link Library Injection, scheduled task persistence, and client exploitation. The selected DLL injection implementation is an authored article using the undocumented `NtCreateThreadEx` API. Its blog URL, C++ source, MinGW compiler, score, and sophistication value remain attached to the stage.

9.4 Compile isolated artifacts

Sources are copied into `samples/a4c6dad3/` and compiled. The session contains nine source outputs and nine binary outputs with zero recorded compilation failures. Inspection with the `host file` utility confirms Windows PE artifacts in the demo set, including x86 and x86-64 executables and DLLs where appropriate.

9.5 Build a defender-facing overlay

All nine stages have at least one Artifact Map association in this snapshot, producing 100% *mapping coverage*. This term is intentionally narrower than detection efficacy. The graph shows, for example, that the selected DLL injection stage maps to 10 Sigma associations and Event IDs 1, 8, 10, and 4688. A defender can open the rule names, compare expected telemetry with lab output, and decide which detections are relevant to the actual environment.

9.6 Interpret, do not overclaim

The session is a threat-informed teaching composition. It is not a claim that Sandworm executed exactly these nine authored samples in exactly this order. The report evidence establishes behavioral relevance; the local module establishes a study implementation; the manifest establishes what Peekaboo built; the detection overlay establishes available rule knowledge. Those are related but distinct claims.

THE COMPLETE PROVENANCE QUESTION

For any node in the graph, ask: What report sentence supports this behavior? Which ATT&CK catalog entry names it? Which authored source demonstrates it? What exactly was compiled? Which telemetry and rules are expected? Peekaboo is designed so those answers are adjacent.

PRACTITIONER EXERCISE

1. Choose one stage and inspect its evidence without looking at the ATT&CK name. Write the behavior in plain language.
2. Compare the selected implementation to the ATT&CK procedure description. Identify what is represented and what is not.
3. Choose two Sigma rules from the overlay. List the telemetry prerequisites that must be enabled before “coverage” can be claimed.
4. Run the artifact only inside an authorized instrumented lab. Compare observed events with the manifest's expected Event IDs.
5. Record mismatches as detection engineering work, not as a reason to change the intelligence evidence.

CHAPTER 10

Evaluation and Readiness Evidence

This evaluation reports what the current snapshot contains and what local checks prove. It does not substitute inventory size for semantic accuracy.

10.1 Database and corpus measurements

DATASET	ROWS	INTERPRETATION
Knowledge documents	320	217 blog documents and 103 book chapters.
Document embeddings	320	Complete nomic - embed - text coverage for current knowledge documents.
Document summaries	320	GPU-precomputed Qwen 14B briefs.
Source TTP extractions	287	Generated source-level ATT&CK mappings.
Curated TTP implementations	145	55 distinct ATT&CK IDs linked to 144 distinct posts.
Artifact Map	410	4,802 Sigma rule associations; 334 rows include Event IDs.
Report TTPs	1,151	Evidence-bearing mappings from 57 distinct report URLs across 21 subjects.
Actor briefs	979	Precomputed local threat profiles.
Family briefs	3,704	Precomputed local malware-family behavior summaries.
Campaign sessions	12	All marked successful and paired with campaign briefs.
Campaign stages	201	Every manifest stage has a corresponding source and binary output; zero compile failures recorded.

10.2 Demo readiness check

The CLI's local doctor `--demo --json` check reported `ready: true`. It verified database integrity and schema availability, 410 ATT&CK Artifact Map rows, 217 blog modules, 145 curated TTP implementations, 1,151 report TTPs, 12 of 12 successful campaign sessions, report data and blog links for all 12, compiled samples for all 12, and availability of GCC, x86-64 MinGW GCC, and NASM.

12/12 CAMPAIGNS SUCCESSFUL	12/12 REPORTS + BLOG LINKS READY	12/12 COMPILED SAMPLES READY
201 MANIFESTED STAGES BUILT	0 RECORDED COMPILE FAILURES	100% ARTIFACT-MAP COVERAGE IN DEMO MANIFESTS

The prepared 12-session set includes Cobalt Strike, Remcos, FormBook, AsyncRAT, NanoCore, Emotet, Agent Tesla, APT41, TA505, APT28, Turla, and Sandworm. The session directories occupy approximately 11 MB and contain 427 files. Every one of the 201 manifest-referenced binary paths exists locally.

10.3 Automated tests

The system Python environment executed 44 tests in 9.529 seconds with no failures; five terminal-UI tests were skipped because Textual was not installed in that interpreter. Covered contracts include stale build output rejection, wall-clock compilation timeouts, source discovery precedence, exact frontend session clearing, local graph dependencies, API security headers, race-safe brief panels, ATT&CK normalization, campaign diversity caps, CLI JSON behavior, offline network blocking, explicit VirusTotal upload consent, closed-pipe behavior, and terminal source-view contracts.

WHAT THESE TESTS PROVE

They prove implementation contracts under the tested environment. They do not prove semantic correctness of every LLM mapping, behavior safety of every authored sample, production SIEM efficacy, or correctness on every compiler and operating-system version.

10.4 Evaluation boundaries

Three numbers require careful interpretation:

1. **100% campaign coverage** means every selected stage joins to at least one Artifact Map row. It does not mean a deployed EDR detected every stage.
2. **1,151 report mappings** measures extracted evidence rows, not 1,151 independently human-verified truths.
3. **201 successful builds** proves the local toolchain produced referenced files. It does not prove every binary executes correctly on every Windows or Linux target.

A stronger future evaluation should execute a stratified sample in instrumented VMs, capture telemetry, measure precision and recall of ATT&CK extraction against human labels, and record whether mapped Sigma rules fire after environment-specific conversion and tuning.

CHAPTER 11

Related Work and Differentiation

Peekaboo is best understood as connective infrastructure. It complements established taxonomies, test libraries, emulation platforms, intelligence catalogs, and detection formats.

11.1 Adjacent systems

MITRE ATT&CK is a knowledge base of adversary behavior based on real-world observations; tactics represent why and techniques represent how [1]. It supplies the common behavioral vocabulary. Peekaboo consumes that vocabulary and attaches it to a private authored corpus, builds, and detections.

Atomic Red Team is a broad library of focused tests mapped to ATT&CK for validating observation and response across Windows, macOS, Linux, and cloud environments [6]. It is stronger as a community test catalog. Peekaboo's differentiator is the first-party article/book relationship and report-to-blog provenance path.

MITRE Caldera automates adversary emulation, supports red teams and incident response, and offers a plugin architecture with agents, C2, reporting, and TTP collections [7]. It is stronger as an operational multi-agent emulation platform. Peekaboo is smaller and research-centric: it focuses on authored source, local compilation, evidence, and detection explanation rather than remote agent orchestration.

Infection Monkey is a breach-and-attack simulation system designed to assess perimeter and internal network resilience through scanning, propagation, and simulated attack behavior [8]. It is stronger for network reachability and propagation assessment. Peekaboo focuses on technique-level source research and detection-oriented campaign composition.

Malpedia is a collaborative malware inventory that provides family, actor, reference, and identification context [3, 4]. Peekaboo uses Malpedia as intelligence input and adds a local bridge to the author's implementation corpus; it does not replace Malpedia's community curation.

Sigma provides a generic, vendor-agnostic representation for log detections [5]. Peekaboo parses Sigma metadata into an ATT&CK-oriented overlay. It does not replace Sigma conversion, rule testing, or SIEM-specific tuning.

11.2 Capability comparison

CAPABILITY	ATT&CK	ATOMIC	CALDERA	MALPEDIA	SIGMA	PEEKABOO
Behavior taxonomy	Core	Uses	Uses	Maps	Tags	Uses
Community executable tests	No	Core	Plugins	No	No	Private corpus
Remote agent orchestration	No	No	Core	No	No	No
Malware family inventory	Software	No	Profiles	Core	No	Integrates
Vendor-neutral detections	Strategies	Guidance	Plugins	YARA	Core	Integrates

CAPABILITY	ATT&CK	ATOMIC	CALDERA	MALPEDIA	SIGMA	PEEKABOO
Private blog/book ingestion	No	No	No	No	No	Core
Report evidence to authored source	No	No	Custom	References	No	Core
GPU bake / offline serve	N/A	N/A	Offline image	No	N/A	Core
Source + build + detection graph	No	Test docs	Operations	No	No	Core

11.3 Where Peekaboo is better suited

“Better” only has meaning relative to an objective. Peekaboo is better suited than a general taxonomy or test repository when the objective is to turn a specific author's long-running research corpus into a traceable teaching and detection workflow. It is better suited than a live LLM dashboard when offline determinism and model privacy are mandatory. It is better suited than a pure report mapper when the analyst wants to open a concrete local implementation and compiled artifact beside the evidence.

Conversely, Caldera is better suited for distributed agent orchestration; Atomic Red Team is better suited for a broad community-maintained test library; Infection Monkey is better suited for propagation and network-path assessment; Malpedia is better suited as the collaborative malware inventory; Sigma is better suited as the portable detection-rule standard; ATT&CK is the authoritative behavioral framework. A credible project explains those boundaries.

11.4 Research context

Automated ATT&CK extraction from threat reports remains an active research problem. Recent work evaluates LLM and hybrid retrieval approaches [9, 10], while a 2025 systematization cautions that automated TTP extraction still has substantial limitations [13]. A July 2026 preprint explores end-to-end CTI-to-Caldera emulation with LLMs [14]. Peekaboo's practical response is modest: constrain the model, persist evidence, validate against a catalog, keep human inspection close, and avoid placing live generation in the execution path.

CHAPTER 12

Technological Contributions

The project's novelty is architectural composition around provenance, not invention of every underlying component.

12.1 The knowledge-to-practice spine

The first contribution is a coherent data path that starts with authored knowledge and ends with a defender-facing experimental object. Existing tools often begin with ATT&CK, a test, an agent, or a rule. Peekaboo begins with a body of explanation and source code, then makes external threat intelligence navigate that body.

12.2 GPU bake, CPU serve

The second contribution is lifecycle-aware use of local AI. Embeddings and generation are treated as build artifacts of the knowledge base. The runtime product is mostly a database reader. This improves privacy, predictability, latency, cost, and demo reliability without hiding the model behind a permanent service dependency.

12.3 Evidence-preserving campaign compilation

The third contribution is the manifest-level preservation of report evidence, ATT&CK behavior, selected authored implementation, source copy, compiler, binary output, and detection overlay. Compilation is therefore not a detached payload-generation action. It is part of a traceable research claim.

12.4 Metadata-derived sophistication

The fourth contribution is a corpus-native answer to example quality. Instead of treating all technique matches equally or hardcoding favorite modules, the selector extracts signals from article series position, title semantics, category, source availability, platform, and compiler metadata. This lets a long-running blog evolve into a graded implementation library.

12.5 Bidirectional ATT&CK bridge

ATT&CK works in two directions. Intelligence can move from report behavior to a local implementation. A blog post or source module can move outward to actor/family relevance, detections, and report evidence. This makes the corpus useful both as a learning library and as a threat-hunting index.

12.6 Deterministic boundaries around probabilistic enrichment

The model does semantic work inside a bounded transformation. Code performs fetching, type detection, text extraction, schema parsing, confidence normalization, tactic selection, deduplication, catalog checks, persistence, stage ordering, build execution, and detection joins. This division is more maintainable than delegating the entire pipeline to an autonomous agent.

PROBABILISTIC

- Semantic similarity
- Prose interpretation
- Technique candidate extraction
- Compact summaries

DETERMINISTIC

- Schema and catalog checks
- Primary tactic ordering
- Module scoring inputs
- Build and manifest contracts
- Detection joins
- Readiness verification

12.7 Campaign graph as a join view

The graph is not decorative visualization. Each node is a database join across intelligence, technique, implementation, artifact, and detection. The Campaign, Hunt, and Evidence modes expose different projections of the same stored session, preserving one source of truth and reducing UI inconsistency.

NOVELTY STATEMENT

Peekaboo's distinctive contribution is a local, authored-corpus-centered, evidence-aware pipeline that compiles threat-informed educational artifacts and places defensive coverage beside them, while using GPU inference as an offline enrichment step rather than a runtime dependency.

CHAPTER 13

Reliability, Safety, and Failure Design

A polished security demo is a system of controlled failure modes: unavailable models, stale URLs, interrupted workers, missing compilers, malformed input, and partial output must remain legible.

13.1 Resumable enrichment

Worker commands write in small increments and use pending-row queries or processed-source records to skip completed work. Interrupt handling prints a resume hint rather than a traceback. This is important when thousands of family briefs or slow report URLs are involved. Resumability also reduces the temptation to create one fragile distributed job for a small local product.

13.2 Offline degradation

Precomputed summaries, actor and family profiles, report TTPs, Artifact Map briefs, campaign briefs, and graphs remain available when Ollama is stopped. Semantic search can report a paused state instead of crashing unrelated screens. Network-sensitive CLI commands honor explicit offline mode, and VirusTotal upload requires affirmative consent in non-interactive use.

13.3 Build correctness contracts

The build manager enforces wall-clock timeouts, rejects stale output from prior runs, snapshots successful outputs per build, infers selected dependencies, and keeps source inputs read-only. Pipeline compilation records stage-local failures rather than destroying the entire campaign. Readiness checks validate required compiler executables before a demonstration.

13.4 Dashboard contracts

Mutation endpoints receive origin checks. Security headers and local frontend dependencies are tested. Session clearing removes database rows and session files. Campaign history responses remain compact. Brief content is shown in one race-safe panel so navigating ATT&CK groups, Technique Library, implementations, and extracted TTPs does not produce overlapping transient UI.

13.5 Dual-use safety

Technical safety cannot be reduced to a banner. Peekaboo helps by keeping work local, making network operations explicit, preserving source provenance, and emphasizing detection. However, many authored examples demonstrate real process injection, persistence, reverse-shell, credential, or evasion concepts. Controls must therefore be operational:

- Use isolated, disposable, instrumented virtual machines.
- Never run samples on production hosts or networks.
- Use benign payloads and loopback or controlled endpoints where possible.

- Require explicit authorization and retain an exercise record.
- Review source before compiling and binaries before execution.
- Treat VirusTotal submission as public disclosure unless using an appropriate private service.
- Keep the dashboard off untrusted networks; it is a research interface, not a hardened internet service.

SECURITY ENGINEERING RULE

A tool that makes offensive research easier must make provenance, consent, isolation, and detection equally visible. Convenience is not a safety boundary.

CHAPTER 14

Deployment and Demo Operations

The recommended workflow prepares intelligence and model outputs on the GPU host, snapshots one database, and proves local readiness on the presentation host.

14.1 Prepare the knowledge plane

```
python3 worker.py scan --book /path/to/book/chapters
python3 worker.py init
python3 worker.py embed --model nomic-embed-text
python3 worker.py tag --model qwen3:1.7b
python3 worker.py ttp --model qwen3:14b
python3 worker.py summarize --model qwen3:14b
```

These commands are incremental. A normal update should process new or changed material rather than erase valid enrichment. Full rebuild flags are appropriate when the model, prompt contract, or source interpretation changes materially.

14.2 Prepare intelligence and detection

```
python3 worker.py reports --actor apt28 --model qwen3:14b
python3 worker.py reports --family win.quasar_rat --model qwen3:14b
python3 worker.py sigma --model qwen3:14b
python3 worker.py actor --model qwen3:14b
python3 worker.py family --model qwen3:14b
python3 worker.py apt --model qwen3:14b
```

Report processing should be curated. Prioritize actors and families with strong accessible references and overlap with authored modules. Persist stale, denied, empty, or low-value source status so the worker can skip them instead of repeatedly spending time.

14.3 Snapshot safely

```
sqlite3 dashboard/peekaboo.db "PRAGMA wal_checkpoint(TRUNCATE);"
sqlite3 dashboard/peekaboo.db ".backup /tmp/peekaboo_snapshot.db"
scp /tmp/peekaboo_snapshot.db demo-host:/path/to/peekaboo/dashboard/peekaboo.db
```

Copying the database is only half of a campaign demo. Session source, binaries, manifests, and generated rules under `samples/` must also be present if the UI should expose downloadable artifacts. The CLI doctor validates this relationship.

14.4 Prove readiness

```
python3 peekaboo_cli.py doctor --demo --json
python3 peekaboo_cli.py status
python3 -m unittest discover -s tests -v
sqlite3 dashboard/peekaboo.db "PRAGMA quick_check;"
```

A useful pre-demo test does not call the internet. It checks database integrity, schema, campaign reports, blog links, sample files, and local toolchains. Keep a second known-good database snapshot and a read-only copy of the prepared sample tree.

14.5 Presentation sequence

1. Open **Malpedia** and identify an actor or family.
2. Show semantically related authored posts and a precomputed brief.
3. Open a prepared **APT Campaign** session.
4. Use **Evidence** mode to show the report sentence and implementation link.
5. Open source in the dashboard or terminal syntax viewer.
6. Show the compiled artifact and manifest.
7. Switch to **Hunt** mode and inspect Sigma/EventID coverage.
8. Finish with the Artifact Map or a generated YARA draft to demonstrate the defensive loop.

DEMO PRINCIPLE

Live interaction should navigate precomputed truth, not create critical truth. Prepare the hard work, then demonstrate the relationships.

CHAPTER 15

Limitations and Research Roadmap

The current system is useful precisely because its boundaries can be stated. The next work should improve evidence quality and measured behavior rather than add more screens.

15.1 Current limitations

SEMANTIC ACCURACY

Language-model output can misclassify behavior or produce plausible but unsupported technique IDs. Evidence requirements reduce but do not remove this risk. The current parent-ID fallback in report validation can admit an unknown sub-technique suffix when the base technique exists. Exact versioned catalog validation is required.

REPORT EXTRACTION COVERAGE

Only two successful sources in the measured batch were identified as PDF by content type; most successful documents were HTML. Scanned PDFs, JavaScript-only sites, anti-bot pages, redirects, and stale vendor URLs remain difficult. OCR and source mirroring are not yet part of the core worker.

DETECTION COVERAGE VERSUS DETECTION EFFICACY

The Artifact Map proves rule associations, not that converted rules fire in a given SIEM or EDR. Event collection, backend translation, field names, exclusions, and baselines are environment-specific. Campaign “coverage” must not be used as an executive efficacy metric without runtime validation.

IMPLEMENTATION FIDELITY

A local blog example may demonstrate the same ATT&CK behavior with a different API, payload, ordering, or operational context than the report. The relationship is educational and behavioral, not source attribution.

PLATFORM BALANCE

The discovery registry has meaningful Linux and macOS content, but the curated TTP implementation table is currently dominated by Windows: 132 Windows rows, 12 macOS rows, and one cross-platform row. Linux implementations should receive explicit curated ATT&CK relationships.

EXECUTION VALIDATION

Compilation is not execution. The prepared artifacts exist and are recognized as PE files where sampled, but a systematic Windows/Linux VM execution matrix with telemetry capture is not yet part of the automated test suite.

15.2 Prioritized roadmap

PRIORITY	WORK	WHY IT MATTERS	PROOF OF COMPLETION
P0	Versioned exact ATT&CK catalog	Eliminates nonexistent sub-techniques and makes taxonomy drift explicit.	Zero report IDs outside the recorded catalog; migration test across versions.
P0	Human review queue for report mappings	Separates generated candidates from accepted campaign evidence.	Reviewer, decision, reason, and timestamp stored; UI filters accepted rows.
P1	Instrumented execution harness	Turns rule association into observed telemetry evidence.	Per-stage VM run record, event capture, hash, exit state, and cleanup proof.
P1	Strict report cache and provenance	Protects against URL rot and content changes.	Content hash, retrieval date, MIME, canonical URL, and local text snapshot.
P1	Curated top-100 report program	Expands useful intelligence without processing the entire low-quality tail.	Quality score, at least one accessible report, accepted TTPs, blog overlap.
P2	IOC extraction and temporal graph	Adds infrastructure and indicator context while preserving evidence.	Typed IOC rows with source sentence, first/last seen, and deduplication.
P2	STIX 2.1 / ATT&CK Navigator export	Improves interoperability with established workflows.	Round-trip schema tests and import into reference tools.
P2	Linux implementation curation	Balances the current Windows-heavy mapping layer.	Curated Linux rows with build and telemetry validation.

15.3 Why not move everything to Elasticsearch or ClickHouse?

For the present 21 MB operational database, SQLite is simpler, portable, and sufficient. A separate report archive of hundreds or thousands of full documents, sentence-level chunks, IOCs, and temporal observations may justify a search or analytical store. The right boundary is to keep the normalized serving contract in SQLite and add an optional external corpus store only when measured query volume, retention, or aggregation requirements exceed it.

A minimal future architecture could store raw and normalized report documents in object storage, index full text and sentence evidence in Elasticsearch, aggregate large behavioral statistics in ClickHouse, and publish only accepted normalized relationships into peekaboo . db. The dashboard would remain simple because the demo path would still read the portable snapshot.

CONCLUSION

Knowledge That Can Be Tested

Peekaboo's main idea is to connect years of authored malware research and book knowledge to the practical security of information systems.

The project turns a private technical corpus into an operational map. Threat reports contribute evidence. ATT&CK contributes a common behavioral language. The blog and book contribute explanation. The meow repository contributes inspectable source. Native toolchains contribute reproducible artifacts. Sigma, Event IDs, and YARA contribute a defender's view. GPU enrichment helps interpret and summarize these layers, then gets out of the runtime path.

This composition is the project's strongest technological contribution. It does not ask an analyst to trust a generated answer in isolation. It provides nearby evidence and objects that can be inspected: the sentence, the ID, the article, the source, the compiler output, the manifest, and the detection rule. Where the system cannot prove a relationship, the roadmap calls for stronger catalog validation, human review, and runtime telemetry rather than more fluent prose.

For workshops, the result is a mini laboratory in which a concept can be read, built, observed, and defended. For threat intelligence, it is a way to navigate from an actor or family into relevant first-party research. For detection engineering, it is a campaign-shaped hunt sheet with artifacts and gaps. For the author, it is a method of making a long-running body of work executable without losing its educational context.

Read → Understand → Map → Build → Observe → Detect → Improve

The system should therefore be judged by a demanding standard: can every important claim be traced, can every build be reproduced, can every detection assumption be tested, and can the entire prepared experience survive without a network or live model? The August 2026 snapshot demonstrates that architecture at meaningful scale. The next step is not to make it larger indiscriminately. It is to make every relationship more exact.



APPENDIX A

Command Reference

A compact operational reference for enrichment, inspection, campaign navigation, and readiness.

Knowledge enrichment

```
python3 worker.py scan [--posts PATH] [--meow-root PATH] [--book PATH]
python3 worker.py init
python3 worker.py embed [--model nomic-embed-text] [--rebuild]
python3 worker.py tag [--model qwen3:1.7b] [--rebuild-changed]
python3 worker.py ttp [--model qwen3:14b]
python3 worker.py summarize [--model qwen3:14b]
python3 worker.py refresh
python3 worker.py status
```

Threat intelligence enrichment

```
python3 worker.py reports --actor ACTOR_ID [--limit N] [--model qwen3:14b]
python3 worker.py reports --family FAMILY_ID [--limit N] [--model qwen3:14b]
python3 worker.py actor [--model qwen3:14b] [--rebuild]
python3 worker.py family [--model qwen3:14b] [--rebuild]
python3 worker.py apt [--model qwen3:14b] [--rebuild]
```

Detection enrichment

```
python3 worker.py sigma --sigma-path /path/to/sigma --rebuild
python3 worker.py sigma --model qwen3:14b
```

CLI operations

```
python3 peekaboo_cli.py doctor --demo --json
python3 peekaboo_cli.py status
python3 peekaboo_cli.py pipeline list
python3 peekaboo_cli.py pipeline show SESSION_ID
python3 peekaboo_cli.py library list
python3 peekaboo_cli.py library show MODULE_ID
python3 peekaboo_cli.py ttp show T1055
python3 peekaboo_cli.py artifacts rules T1055
python3 peekaboo_cli.py --offline malpedia search QUERY
python3 peekaboo_cli.py tui
```

Database snapshot

```
sqlite3 dashboard/peekaboo.db "PRAGMA quick_check;"  
sqlite3 dashboard/peekaboo.db "PRAGMA wal_checkpoint(TRUNCATE);"  
sqlite3 dashboard/peekaboo.db ".backup /tmp/peekaboo_snapshot.db"
```

Test suite

```
python3 -m unittest discover -s tests -v
```

APPENDIX B

Data Model

The principal tables and the claims they are allowed to make.

TABLE	KEY IDENTITY	ROLE
kb_docs	Document slug	Normalized blog and book metadata, source path, category, editorial ATT&CK IDs.
kb_embeddings	Document + model	Stored vectors for semantic retrieval.
kb_tags	Document + model	Generated classification tags; model generations can coexist.
ttp_extracted	Document + model	Generated source-level ATT&CK candidates with rationale/confidence.
kb_summaries	Document + model	Precomputed compact knowledge briefs.
ttp_implementations	ATT&CK + blog slug	Curated relationship between a technique and authored implementation.
artifact_map	ATT&CK ID	Sigma-derived rule count, Event IDs, registry, process, command-line, and rule metadata.
artifact_summaries	ATT&CK + model	Precomputed detection brief.
report_ttp_sources	Subject + URL + model	Report processing status, content type, text size, error, and timestamp.
report_ttps	Subject + report + model + TTP + evidence hash	Evidence-bearing generated report mapping.
actor_summaries	Actor + model	GPU-precomputed actor profile brief.
family_summaries	Family + model	GPU-precomputed malware-family behavior brief.
pipeline_sessions	Session ID	Actor/family, timestamps, status, extracted TTP set, and complete stage parameters.
session_summaries	Session + model	GPU-precomputed campaign brief.
samples	Session ID	Filesystem outputs, size, subject, TTP count, and build status.
query_embeddings	Query + model	Cache for the one optional runtime model operation.

Trust labels recommended for future UI

LABEL	MEANING	EXAMPLE
Editorial	Directly authored or manually curated.	Blog frontmatter ATT&CK mapping; curated implementation row.
Extracted	Generated from source or prose and normalized by code.	Report TTP with evidence.
Derived	Computed by deterministic joins or aggregation.	Campaign coverage and unique Event IDs.
Built	Observed as a local build output.	Source copy, binary path, size, compiler result.
Validated	Checked against an explicit catalog or runtime observation.	Exact ATT&CK ID, VM telemetry event.
Reviewed	Accepted or rejected by a named human.	Approved report mapping.

APPENDIX C

Glossary

TERM	MEANING IN THIS PAPER
ATT&CK	MITRE's knowledge base and taxonomy of adversary tactics, techniques, and procedures.
Artifact Map	Peekaboo's per-technique aggregation of Sigma rules, Event IDs, registry keys, processes, and command-line indicators.
Baked output	A model-derived result computed before serving and persisted in SQLite.
Brief	A compact model-generated summary constrained for one UI context.
Campaign	A persisted Peekaboo session that composes report-derived TTPs, local implementations, artifacts, and detections.
Coverage	In current manifests, the proportion of stages with Artifact Map associations; not measured EDR efficacy.
Evidence	A short report sentence or excerpt stored beside an extracted ATT&CK mapping.
GPU enrichment	Local model inference that creates embeddings, structured extractions, and briefs. It is not base-model training.
Implementation	An authored source example connected to an ATT&CK technique.
Manifest	The session JSON record describing stages, provenance, build outputs, and detection overlay.
Malpedia	A collaborative malware inventory used as a source of actor, family, and report-reference context.
Provenance	Recorded origin and transformation history of a claim or artifact.
Semantic search	Vector similarity retrieval over the local knowledge corpus.
Sigma	A generic, vendor-neutral format for describing log detection logic.
Sophistication score	A metadata-derived signal that biases module selection toward deeper authored examples.
TTP	Tactic, technique, and procedure; used here primarily for ATT&CK-mapped behavior.
YARA	A pattern-based language and engine for identifying files or memory content.

REFERENCES

Sources and Further Reading

1. MITRE. "Get Started with ATT&CK." <https://attack.mitre.org/resources/>. ATT&CK is described as a knowledge base of adversary techniques based on real-world observations.
2. MITRE. "ATT&CK Updates, April 2026, Version 19." <https://attack.mitre.org/resources/updates/>.
3. D. Plohmann, M. Clauss, S. Enders, and E. Padilla. "Malpedia: A Collaborative Effort to Inventorize the Malware Landscape." *The Journal on Cybercrime & Digital Investigations*, 3(1), 2018. Citation details at [Malpedia References](#).
4. Fraunhofer FKIE. "Malpedia Website Usage and Family Details." <https://malpedia.caad.fkie.fraunhofer.de/usage/website>.
5. SigmaHQ. "Sigma: Generic Signature Format for SIEM Systems." <https://sigmahq.io/sigma/> and Sigma Specification.
6. Red Canary. "Atomic Red Team." <https://redcanary.com/atomic-red-team/>.
7. MITRE. "Caldera: Automated Adversary Emulation Platform." <https://github.com/mitre/caldera>.
8. Akamai / Guardicore. "Infection Monkey Documentation." <https://techdocs.akamai.com/infection-monkey/docs/welcome-infection-monkey>.
9. "Towards Effective Identification of Attack Techniques in Cyber Threat Intelligence Reports using Large Language Models." arXiv:2505.03147, 2025. <https://arxiv.org/abs/2505.03147>.
10. "MITREtrieval: Retrieving MITRE Techniques From Unstructured Threat Reports by Fusion of Deep Learning and Ontology." *IEEE Transactions on Network and Service Management*, 2024. <https://doi.org/10.1109/TNSM.2024.3401200>.
11. Cytoscape Consortium. "Cytoscape.js Documentation." <https://js.cytoscape.org/>.
12. VirusTotal. "Files API." <https://docs.virustotal.com/reference/files>.
13. V. Buechel et al. "SoK: Automated TTP Extraction from CTI Reports: Are We There Yet?" *34th USENIX Security Symposium*, 2025. [USENIX paper page](#).
14. "Fully Automated End-to-End Adversary Emulation from MITRE ATT&CK Based Cyber Threat Intelligence Using LLMs." arXiv:2607.14566, 2026. <https://arxiv.org/abs/2607.14566>.
15. Ollama. "Generate Embeddings API." <https://docs.ollama.com/api/embed>.
16. YARA. "Writing YARA Rules." <https://yara.readthedocs.io/en/stable/writingrules.html>.
17. SQLite. "Write-Ahead Logging." <https://sqlite.org/wal.html>.
18. Zhassulan Zhussupov. "cocomelonc's blog." <https://cocomelonc.github.io/>; source corpus: <https://github.com/cocomelonc/meow>.
19. Zhassulan Zhussupov. "Peekaboo." <https://github.com/cocomelonc/peekaboo>.

Measurement note. All project counts in this paper come from the local repository, SQLite database, filesystem manifests, compiler inspection, readiness command, and test suite on 1 August 2026. External sources are used for definitions and related-work positioning. This document is a project technical paper and practitioner guide; it has not undergone academic peer review.